UDC 004.89

AN AGENTIC RAG SYSTEM FOR YOUTUBE VIDEO ANALYSIS VIA A TELEGRAM BOT INTERFACE

Roman Kuliievych[1]

[1]Lviv Polytechnic National University, Department of Information Systems and Networks, Lviv, Ukraine

[1] E-mail: roman.kuliievych.mnsam.2025@lpnu.ua, ORCID: 0009-0003-7266-4607

© Kuliievych Roman, 2026

Abstract. *The rapid growth of educational and informational video content on YouTube makes it increasingly difficult for users to locate specific facts within long recordings without watching them in full. The paper addresses the problem of automated, conversational question answering over the content of YouTube videos through a familiar messenger interface. The purpose of the study is to design, implement and experimentally evaluate an agentic Retrieval-Augmented Generation (RAG) system, accessible via a Telegram bot, that ingests automatically generated video transcripts, indexes them in a vector store, and answers user questions in natural language, in the language of the source video. The proposed architecture combines an LLM-based query router that decides whether a question can be answered from the short-term dialogue context or requires a new database search, hybrid dense-sparse retrieval fused with Reciprocal Rank Fusion (RRF), a cross-encoder reranking stage, and a generation module that maintains a five-message dialogue history. Both a local (Ollama) and a cloud (AWS Bedrock) deployment configuration are supported. An experimental evaluation of four transcript-chunking strategies - sentence-based chunking with one- and thirty-token overlap, timestamp-based chunking, and semantic chunking - was carried out on ten questions of five types (factual, summary, multi-hop, follow-up, off-topic) for each of three videos, using RAGAS-style metrics: faithfulness, answer relevancy, context precision and context recall. The results show that sentence-based chunking with a thirty-token overlap achieves the highest composite quality score (0.871) among the tested configurations, at comparable latency and cost, while off-topic questions remain the hardest category regardless of the chunking strategy used. The scientific novelty lies in the systematic, metric-based comparison of chunking strategies for conversational video question answering, combined with an LLM-based context/database routing mechanism that reduces redundant retrieval. The practical significance is a reproducible, cloud-portable architecture that can be reused for building similar retrieval-augmented conversational assistants over long-form video or audio content.*

Keywords - retrieval-augmented generation, large language models, YouTube transcripts, Telegram bot, hybrid search, chunking strategies, conversational agent.

INTRODUCTION. THE PROBLEM STATEMENT

YouTube has become one of the largest repositories of educational, technical and analytical video content, with individual lecture, tutorial, and interview videos frequently exceeding one hour in length. Users who need a specific fact, definition, or opinion expressed at an unknown point in such a video have no efficient way to locate it other than watching or manually scrubbing through the recording, or relying on a full-text search of the video description, which rarely covers the spoken content. At the same time, large language models (LLMs) have demonstrated a strong ability to answer natural-language questions, but when used without access to the source material they are prone to hallucination and cannot reference video-specific facts that were never part of their training data.

Retrieval-Augmented Generation (RAG) addresses this limitation by grounding LLM answers in externally retrieved evidence, which is particularly suitable for video transcripts: a transcript can be automatically obtained, split into chunks, embedded, indexed, and searched at query time, so that the LLM only has to synthesize an answer from the retrieved fragments rather than from its parametric memory. Delivering such

a system through a Telegram bot removes any need for a dedicated application, since the majority of target users - students, researchers, and industry professionals - already use instant messengers on a daily basis.

The problem addressed in this study is therefore twofold: (1) how to design an agentic RAG pipeline that reliably answers multi-turn, conversational questions about a specific YouTube video, including follow-up and multi-hop questions, while correctly rejecting off-topic questions; and (2) which transcript-chunking strategy provides the best trade-off between answer quality, latency, and inference cost for automatically generated, timestamped captions that are noisier and less structured than conventional documents.

ANALYSIS OF RECENT RESEARCH AND PUBLICATIONS

Retrieval-Augmented Generation was introduced as a way of combining a parametric generator with a non-parametric retriever over an external knowledge index, which reduces hallucination and allows the underlying knowledge base to be updated without retraining the model (Lewis et al., 2020). Subsequent work formalized dense retrieval using bi-encoder embeddings trained specifically for question answering (Karpukhin et al., 2020) and general-purpose sentence embedding models suitable for semantic search (Reimers & Gurevych, 2019). A comprehensive survey of RAG architectures classifies systems into naive, advanced and modular paradigms and highlights hybrid retrieval, reranking and query routing as the main levers for improving answer quality (Gao et al., 2024).

Combining sparse lexical retrieval (e.g., BM25) with dense vector retrieval is known to outperform either method individually; Reciprocal Rank Fusion (RRF) provides a simple, parameter-light way of merging ranked lists from heterogeneous retrievers without requiring score normalization (Cormack, Clarke, & Buettcher, 2009), which makes it attractive for production systems that mix embedding-based and keyword-based search. Cross-encoder and LLM-based rerankers are commonly added as a second-stage filter to improve the precision of the top-k retrieved passages before they are passed to the generator (Gao et al., 2024).

Agentic patterns that let an LLM decide between several possible actions - for example, answering directly, calling a tool, or issuing a new retrieval query - have been shown to improve reasoning and reduce unnecessary tool calls compared with a fixed, always-retrieve pipeline (Yao et al., 2023). This routing idea is directly relevant to conversational RAG bots, where many follow-up questions can be answered from the previous turn's retrieved context without an additional, costly database search.

Evaluating RAG systems is itself an open research problem, since traditional QA metrics (exact match, BLEU/ROUGE) do not capture whether an answer is actually grounded in the retrieved evidence. The RAGAS framework proposes reference-free metrics - faithfulness, answer relevancy, context precision and context recall - computed with an auxiliary LLM judge, which allows large-scale automated evaluation without manually annotated gold answers for every question (Es et al., 2024).

Video-specific question answering has mostly been studied for narrated instructional videos with an emphasis on multimodal (visual + audio) grounding (Yang, Miech, Sivic, Laptev, & Schmid, 2021), whereas the specific problem of chunking automatically generated, timestamp-aligned captions for a purely text-based RAG pipeline, and of routing between conversational memory and fresh retrieval inside a messenger bot, has received comparatively little attention in the recent literature. In addition, most published evaluations of chunking strategies are conducted on curated text corpora rather than on noisy, punctuation-poor YouTube auto-captions, and rarely report cost and latency alongside answer quality. This gap motivates the present study, which evaluates chunking strategies specifically for conversational YouTube-transcript question answering and reports quality, cost and latency jointly.

MATERIALS AND METHODS

The developed system implements an agentic RAG pipeline with two logically separate stages: an offline ingestion pipeline that prepares a searchable index for a given video, and an online query pipeline that answers user questions inside a Telegram chat. The overall architecture is shown in Fig. 1.

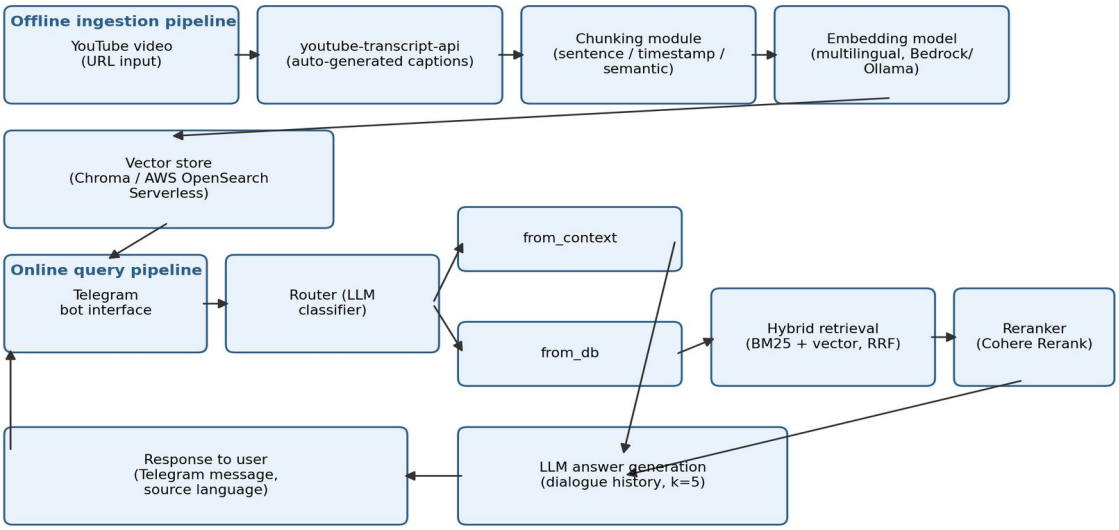


Fig. 1. Architecture of the YouTube RAG Telegram bot (developed by the author)

Transcript ingestion. When a user submits a YouTube link, the system fetches the automatically generated captions of the video using the youtube-transcript-api library. No fallback speech-recognition (ASR) stage is implemented in the current version; if the video has no automatically generated or manual captions, transcript extraction fails and the video cannot be indexed. The obtained transcript, together with per-segment timestamps, is passed to a chunking module.

Chunking strategies. Four chunking strategies were implemented and compared: (1) sentence-based chunking with a target size of 300 tokens and one sentence of overlap between consecutive chunks; (2) sentence-based chunking with the same target size but a thirty-token overlap; (3) timestamp-based chunking, which additionally stores the start/end timestamps of each segment in the chunk metadata and splits text on sentence boundaries within a 300-token window with a one-sentence overlap; and (4) semantic chunking, which merges consecutive sentences into a single, non-overlapping chunk while the cosine similarity between the sentence embedding and the running chunk embedding stays above a threshold of 0.75, so that each chunk is topically coherent and no two chunks share text.

Embedding and indexing. Chunks are embedded with a multilingual embedding model (Cohere embed-multilingual-v3 in the cloud configuration, nomic-embed-text in the local configuration), which allows the system to answer English-language questions about videos in other languages by matching semantically rather than lexically. Depending on the deployment target, vectors are stored either in a local Chroma instance or in an AWS OpenSearch Serverless collection, accessed through an S3-staged, SQS-driven asynchronous ingestion pipeline that decouples transcript processing from the Telegram bot's request/response cycle.

Query routing. Every incoming question is first classified by an LLM-based router into one of two routes: from_context, when the question can be fully answered from the previous turn's retrieved context (e.g., short follow-up questions), or from_db, when a fresh retrieval over the vector index is required. The router additionally receives the last five messages of the dialogue history so that pronoun references and short follow-up questions can be resolved correctly. This routing step avoids an unnecessary retrieval call for a substantial share of the traffic and keeps the dialogue coherent across turns.

Hybrid retrieval and reranking. For questions routed to from_db, the system retrieves the top similarity_k = 10 candidates independently from a dense vector search and a sparse BM25 search, and fuses the two ranked lists with Reciprocal Rank Fusion:

$$score(d) = \Sigma\ 1 / (k + rank^{i}(d)) \quad (1)$$

where d is a candidate document (chunk), the sum is taken over every ranked list in which d appears, rank_i(d) is its rank (starting from 1) in list i, and k = 60 is the standard damping constant that limits the influence of any single very high rank. The fused list is then narrowed to the top retrieval_top_k = 4 chunks by a Cohere Rerank v3.5 cross-encoder reranker, which reorders candidates by their fine-grained relevance to the exact question text before they are inserted into the generation prompt.

Answer generation. The reranked chunks, the last five dialogue turns, and the user's question are passed to an LLM (amazon.nova-lite-v1:0 in the Bedrock configuration, gemma3:4b in the local Ollama configuration) that produces the final answer in the language of the source video, in accordance with the multilingual retrieval design. The answer is returned to the user as a Telegram message.

Evaluation methodology. To compare the four chunking strategies, three YouTube videos in English were indexed, and a fixed set of ten questions per video (thirty questions in total) was designed, covering five question types: factual, summary, multi-hop, follow-up, and off-topic (a question unrelated to the video content, used to test whether the system correctly avoids fabricating an answer). Each of the thirty questions was submitted once per chunking strategy, giving 4 x 30 = 120 recorded runs. For every run the system logged the total response latency, the retrieval latency, the number of input/output LLM tokens, the estimated cost in USD, the routing decision, and four RAGAS-style quality metrics computed by an LLM judge: faithfulness (is the answer supported by the retrieved context), answer relevancy (does the answer address the question), context precision (how much of the retrieved context is relevant) and context recall (how much of the necessary information was retrieved). A composite quality_score was defined as the mean of the four available RAGAS metrics for each run, ignoring missing values.

RESEARCH GOALS

The objective of this study is to design an agentic RAG architecture for conversational question answering over YouTube video transcripts, deployable through a Telegram bot in both a local and a cloud configuration, and to experimentally determine which transcript-chunking strategy provides the best balance of answer quality, retrieval cost, and latency for this task. A secondary objective is to identify which categories of user questions remain the hardest for the system to answer correctly, in order to guide future improvements of the routing and retrieval components.

RESULTS AND DISCUSSION

Table 1 presents the mean values of the four RAGAS metrics and the composite quality_score for each of the four chunking strategies, averaged over the thirty questions evaluated per strategy.

Table 1.

Mean RAGAS metrics by chunking strategy (n = 30 questions per label)

Chunking strategy	Faithfulness	Answer relevancy	Context precision	Context recall	Quality score
sentence 300/30	0.9571	0.7480	0.8565	0.9222	0.8710
sentence 300/1	0.9678	0.7609	0.8407	0.9056	0.8688
semantic 300 (thr 0.75)	0.9842	0.6903	0.8231	0.8889	0.8466

timestamp 300/1	0.9465	0.7668	0.8074	0.8222	0.8355

Source: developed by the author, based on 120 experimental runs.

Sentence-based chunking with a thirty-token overlap achieved the highest composite quality_score (0.871), narrowly ahead of the one-token-overlap variant (0.869); the difference between the two sentence-based configurations is small relative to their standard deviation (~0.15), which suggests that the benefit of a larger overlap is modest for this dataset. Semantic chunking obtained the highest faithfulness (0.984), consistent with the fact that its chunks are topically coherent and contain no duplicated content that could confuse the generator, but its answer relevancy was the lowest of the four strategies (0.690), and its context recall was also the lowest (0.889), which indicates that non-overlapping, similarity-driven chunk boundaries occasionally split relevant information across chunks that are not jointly retrieved. Timestamp-based chunking obtained the lowest overall quality_score (0.836), driven mainly by low context precision (0.807) and context recall (0.822), suggesting that segmenting strictly by timestamp boundaries does not always align with the topical boundaries of the discussion.

Fig. 2 visualizes the same ranking together with the variability (standard deviation) of the composite score across question types.

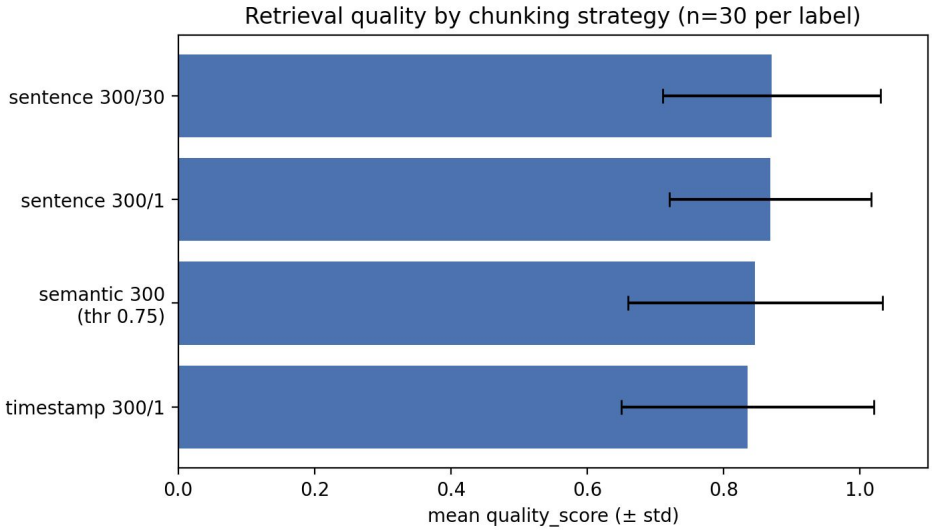


Fig. 2. Overall quality ranking of chunking strategies, mean ± std (developed by the author)

Table 2 breaks the composite quality_score down by question type, averaged over all four chunking strategies, and shows which type of question is the hardest for the system overall.

Table 2.

Mean quality score by question type (all chunking strategies pooled)

Question type	Faithfulness	Answer relevancy	Context precision	Context recall	Quality score	n
off_topic	0.9792	0.5117	0.3333	0.4167	0.5602	12
summary	0.9934	0.7539	0.7542	0.9000	0.8500	20
follow_up	0.9538	0.7236	0.9236	0.8750	0.8690	24
multi_hop	0.9612	0.8893	0.8437	0.9236	0.9045	24
factual	0.9534	0.7264	0.9583	1.0000	0.9093	40

Source: developed by the author.

Off-topic questions are, by a wide margin, the hardest category for every chunking strategy (mean quality_score 0.560, versus 0.850-0.910 for the other four types), mainly because of low context precision (0.333) and context recall (0.417): the retriever still returns chunks with the highest available similarity score even when no chunk is actually relevant, and the generator does not always recognize that the retrieved context does not answer the question. This indicates that the router and the generation prompt should be strengthened with an explicit "no relevant information found" fallback rather than relying only on the retrieved evidence. Factual and multi-hop questions are handled best (quality_score 0.909 and 0.905, respectively), which confirms that the hybrid retrieval and reranking stages are effective at surfacing directly stated facts and at combining evidence from more than one chunk.

Cost and latency were nearly identical across the four strategies: the average cost per question ranged only from $0.002142 (semantic) to $0.002153 (timestamp), and average total latency ranged from 2058 ms (sentence 300/30) to 2168 ms (timestamp 300/1), because all four strategies produce a similar number of chunks of a similar target size and therefore a similar prompt length. The correlation between quality_score and cost, latency, or token counts was weak and statistically negligible (Pearson r between -0.02 and 0.14 for all four variables), confirming that, for this task, chunking strategy - not spending more tokens or more time - is the dominant lever for answer quality. A composite ranking that combines quality (weight 0.7), cost (weight 0.2) and latency (weight 0.1) also selects sentence-based chunking with a thirty-token overlap as the best overall configuration.

The query router sent 76.7-80.0% of questions to a fresh database search (from_db) and the remaining 20.0-23.3% to the conversational context (from_context), depending on the chunking strategy. Questions answered from context achieved a higher mean quality_score than questions requiring a new retrieval (0.876 versus 0.850), which supports the design decision to route short follow-up questions to the existing context whenever possible, both for answer quality and for reducing unnecessary retrieval calls and the associated latency and cost.

Limitations and directions for further measurement. The experiment reported here isolates the effect of the transcript-chunking strategy; it does not yet compare the local (Ollama) and cloud (AWS Bedrock) deployment configurations, which is expected to be the main source of latency and infrastructure-cost differences in practice. A minimal follow-up measurement protocol is therefore proposed: (1) repeat the same 10-15 question set on 3 videos for both providers and record end-to-end response latency, indexing latency per minute of video, peak memory consumption, and the actual USD cost per question; (2) increase the number of repeats per question to at least 3 in order to quantify the stability of the RAGAS metrics, which in the present dataset already show a moderate standard deviation (up to 0.19) that a single repeat cannot separate from genuine question difficulty; and (3) extend the question set to multilingual videos to validate the cross-lingual retrieval and generation behavior claimed by the architecture. These measurements were outside the scope of the present study and are left for future work.

CONCLUSIONS

This paper presented the design and an experimental evaluation of an agentic Retrieval-Augmented Generation system that answers conversational questions about YouTube videos through a Telegram bot interface. The proposed architecture combines automatic transcript ingestion, four alternative chunking strategies, an LLM-based context/database router, hybrid dense-sparse retrieval fused with Reciprocal Rank Fusion, a cross-encoder reranking stage, and a multilingual, history-aware generation module, deployable both locally (Ollama) and in the cloud (AWS Bedrock).

An evaluation of 120 question-answer runs across three videos, five question types and four chunking strategies showed that sentence-based chunking with a thirty-token overlap achieves the best overall trade-off between faithfulness, relevancy, context quality, cost and latency (composite quality_score 0.871), while semantic chunking maximizes faithfulness at the cost of answer relevancy, and timestamp-based

chunking underperforms on context precision and recall. Off-topic questions were confirmed as the hardest category for the system regardless of the chunking strategy, pointing to the router and the generation prompt, rather than the retriever configuration, as the priority areas for improvement.

The scientific novelty of the work consists in the systematic, metric-based comparison of transcript-chunking strategies specifically for conversational, timestamp-aligned YouTube captions, jointly with an LLM-based routing mechanism between conversational memory and fresh retrieval. The practical significance lies in a reproducible and cloud-portable reference architecture that can be adopted for similar retrieval-augmented conversational assistants over long-form audio or video content. Future work will extend the evaluation to a direct comparison of the local and cloud deployment stacks in terms of latency, cost and resource consumption, following the minimal measurement protocol outlined above, and will investigate an explicit relevance-gating mechanism to reduce hallucination on off-topic questions.

ACKNOWLEDGEMENTS

The author would like to thank Dr. Dmytro Dosyn, the scientific supervisor of this work, for his guidance and valuable advice during the design and evaluation of the system described in this paper.

Conflict of Interest

The author declares that there is no conflict of interest.

AUTHOR CONTRIBUTIONS

The author confirms sole responsibility for the study conception and design, software implementation, data collection, analysis, interpretation of results, and manuscript preparation.

REFERENCES

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (pp. 758-759). https://doi.org/10.1145/1571941.1572114

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (pp. 150-158). Association for Computational Linguistics.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. arXiv. https://doi.org/10.48550/arXiv.2312.10997

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (pp. 6769-6781). https://doi.org/10.18653/v1/2020.emnlp-main.550

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (pp. 3982-3992). https://doi.org/10.18653/v1/D19-1410

Yang, A., Miech, A., Sivic, J., Laptev, I., & Schmid, C. (2021). Just ask: Learning to answer questions from millions of narrated videos. In Proceedings of the IEEE/CVF International Conference on Computer Vision (pp. 1686-1697). https://doi.org/10.1109/ICCV48922.2021.00171

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In Proceedings of the 11th International Conference on Learning Representations.

АГЕНТ ДЛЯ АНАЛІЗУ ВІДЕО З YOUTUBE ЗА ДОПОМОГОЮ RAG-СИСТЕМИ ЧЕРЕЗ ІНТЕРФЕЙС ТЕЛЕГРАМ-БОТА

Кулієвич Роман Михайлович[1]

[1]Національний університет “Львівська політехніка”, кафедра інформаційних систем та мереж, Львів, Україна

[1] E-mail: roman.kuliievych.mnsam.2025@lpnu.ua, ORCID: 0009-0003-7266-4607

© Кулієвич Роман, 2026

Анотація. *Стрімке зростання обсягів освітнього та інформаційного відеоконтенту на YouTube ускладнює для користувачів пошук конкретних фактів у тривалих відео без їх повного перегляду. У статті розглянуто проблему автоматизованої, діалогової відповіді на запитання щодо змісту відео з YouTube через звичний інтерфейс месенджера. Метою дослідження є розроблення, реалізація та експериментальне оцінювання агентної системи Retrieval-Augmented Generation (RAG), доступної через телеграм-бота, яка отримує автоматично згенеровані субтитри відео, індексує їх у векторному сховищі та відповідає на запитання користувача природною мовою, якою озвучено вихідне відео. Запропонована архітектура поєднує маршрутизатор запитів на основі LLM, який визначає, чи можна відповісти на запитання з контексту попереднього діалогу, чи потрібен новий пошук у базі даних, гібридний щільно-розріджений пошук, об'єднаний методом Reciprocal Rank Fusion (RRF), етап переранжування на основі крос-енкодера та модуль генерації відповіді, що враховує історію діалогу з п'яти повідомлень. Реалізовано локальну (Ollama) та хмарну (AWS Bedrock) конфігурації розгортання. Проведено експериментальне порівняння чотирьох стратегій чанкінгу транскриптів - посегментного чанкінгу з перекриттям в одне та тридцять токенів, чанкінгу за таймкодами та семантичного чанкінгу - на десяти запитаннях п'яти типів (фактичні, узагальнювальні, багатоетапні, уточнювальні та ті, що не стосуються теми) для кожного з трьох відео, з використанням метрик у стилі RAGAS: достовірність, релевантність відповіді, точність контексту та повнота контексту. Результати показують, що посегментний чанкінг із перекриттям у тридцять токенів забезпечує найвищу композитну оцінку якості (0,871) серед перевірених конфігурацій за порівнянних вартості й затримки, тоді як запитання не по темі залишаються найскладнішою категорією для всіх конфігурацій. Наукова новизна полягає в системному, метричному порівнянні стратегій чанкінгу для діалогової системи запитань-відповідей за відеотранскриптами у поєднанні з механізмом маршрутизації на основі LLM. Практичне значення отриманих результатів полягає у можливості використання відтворюваної, переносної між середовищами архітектури для побудови подібних діалогових асистентів на основі RAG.*

Ключові слова - генерація з розширенням пошуком, великі мовні моделі, транскрипти YouTube, телеграм-бот, гібридний пошук, стратегії чанкінгу, діалоговий агент.